

Primary Key and Unique Key on Same Column in a Table Issue -- SQL Server

Issue:

```
CREATE TABLE [HR].[EmpTbl]
(
    [id] [nvarchar](40) NOT NULL,
    [date] [date] NOT NULL,
    [Employee] [nvarchar](40) NOT NULL,

    CONSTRAINT [PK_EmpTbl_id]
    PRIMARY KEY CLUSTERED ( [id] ASC ),

    CONSTRAINT [EmpTbl_id_uindex]
    UNIQUE NONCLUSTERED ( [id] ASC )
)
```

- Krishna Reddy has noticed above 2 constraints while doing index tuning activity in their old Prod DB and each constraint associated with one index.
- Like this there are dozens of tables contains primary key and unique key on SAME COLUMN in their database.
- He believes unique key constraint is unnecessary in this case as prime key already enforcing uniqueness on the column.
- Is there any specific reason behind creating unique key constraint in this case?
- Have you ever encountered such case in your DBA experience?

Query from: [krishna reddy](#)

Source: https://www.linkedin.com/posts/krishna-reddy-b78589197_hello-dbas-mssql-create-table-hremptbl-activity-7379088949471825920-i-AM?utm_source=share&utm_medium=member_desktop&rcm=ACoAACKCDIIBwCyHwI-zdzyASa1BIP491KNZCLE

Situation Recap:

Krishna Reddy found multiple tables in their production MSSQL database where:

- A **primary key constraint** is defined on a column (e.g., [id]).
- A **unique constraint** is also defined on **the same column** (e.g., [id]).

And yes — **each of these constraints creates a separate index** (unless suppressed, which they're not in your case).

Key Facts About Constraints and Indexes in SQL Server:

Constraint Type	Enforces	Creates Index?	Notes
PRIMARY KEY	Uniqueness + NOT NULL	Yes, by default it creates a clustered or non-clustered index	
UNIQUE	Uniqueness only	Yes, creates a non-clustered index unless specified otherwise	

In his case:

PRIMARY KEY CLUSTERED ([id] ASC)

UNIQUE NONCLUSTERED ([id] ASC)

This means:

- A **clustered index** exists on [id] from the primary key.
- A **non-clustered unique index** also exists on the **same column**.

This **duplicates effort and consumes space** — and in almost all cases, it's **unnecessary**.

Why might someone have done this?

Possible reasons (some valid, some not):

1. **Lack of understanding** — someone may not have realized the primary key already enforces uniqueness.
2. **Tool-generated code** — some ORMs, database design tools, or old codegen tools may auto-generate both constraints blindly.
3. **Indexing strategy mismatch** — someone may have tried to **force a specific type of index** (clustered vs non-clustered) but didn't realize the constraint already did the job.
4. **Copy-paste habit** — redundant constraints may have spread across the schema via reused scripts or templates.
5. **Historical refactoring** — the unique constraint might have existed before the primary key was added, and never cleaned up.

Why it's problematic:

- **Redundant storage:** Extra index takes up disk space.
- **Extra overhead on DML:** Inserts/updates/deletes need to maintain two indexes instead of one.
- **Wasted maintenance cycles:** Index rebuilds/reorgs/stats updates on unused indexes.
- **Index tuning confusion:** Makes index tuning harder due to unnecessary indexes cluttering the view.

Best Practice:

 If the primary key is already defined on a column, there is no need to define a separate unique constraint on the same column.

You can safely **drop the unique constraint** — assuming:

- It is **not being explicitly referenced** by any code, foreign keys, or triggers.
- The column is not being queried using that specific **non-clustered index** for performance in ways that differ from the clustered index.

🔗 How to find and clean them:

Find all such duplicate constraints:

SELECT

```
t.name AS TableName,  
c.name AS ColumnName,  
pk.name AS PrimaryKeyName,  
uq.name AS UniqueKeyName
```

FROM

```
sys.tables t
```

JOIN

```
sys.indexes pk ON t.object_id = pk.object_id AND pk.is_primary_key = 1
```

JOIN

```
sys.index_columns pkc ON pk.object_id = pkc.object_id AND pk.index_id = pkc.index_id
```

JOIN

```
sys.columns c ON pk.object_id = c.object_id AND pkc.column_id = c.column_id
```

JOIN

```
sys.indexes uq ON t.object_id = uq.object_id AND uq.is_unique_constraint = 1 AND uq.index_id <> pk.index_id
```

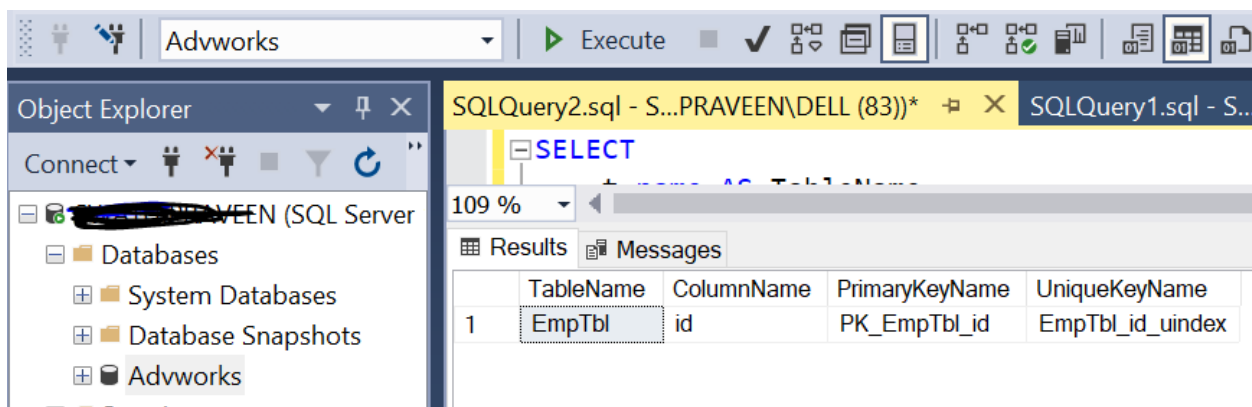
JOIN

```
sys.index_columns uqc ON uq.object_id = uqc.object_id AND uq.index_id = uqc.index_id AND uqc.column_id = pkc.column_id
```

WHERE

```
pkc.key_ordinal = 1  
AND uqc.key_ordinal = 1
```

Example: Tested mentioned table on Advworks database and ran above script for verification.



The screenshot shows the SQL Server Enterprise Manager interface. The Object Explorer on the left shows the Advworks database. The SQL Query window on the right displays the results of a query. The query is a SELECT statement that finds duplicate constraints. The results table has four columns: TableName, ColumnName, PrimaryKeyName, and UniqueKeyName. The results show one row for the EmpTbl table, column id, with primary key PK_EmpTbl_id and unique key EmpTbl_id_uindex.

	TableName	ColumnName	PrimaryKeyName	UniqueKeyName
1	EmpTbl	id	PK_EmpTbl_id	EmpTbl_id_uindex

To drop the redundant unique constraint:

1. First, **find its name** (e.g., EmpTbl_id_uindex)
2. Then drop:

ALTER TABLE HR.EmpTbl DROP CONSTRAINT EmpTbl_id_uindex;

✅ Final Takeaway:

Its right — the **unique constraint on the same column as the primary key is redundant and unnecessary**, and removing it can reduce maintenance and improve performance. It's likely legacy or tool-generated, not intentional design.

Here's a **safe, script-based approach** to:

1.  **Identify** all unique constraints on the *same column* as a primary key (i.e., redundant).
2.  **Generate ALTER TABLE ... DROP CONSTRAINT statements** for them — **without auto-executing**, so you can review them first.
3.  You can optionally execute them after review.

Step 1: Generate DROP Statements for Redundant Unique Constraints (On a Specific Table)

-- This script generates DROP statements for unique constraints that duplicate primary key constraints on the same column(s) on a SPECIFIC Table:

```
SELECT top 1
    sch.name AS SchemaName,
    t.name AS TableName,
    uc.name AS UniqueConstraintName,
    'ALTER TABLE [' + sch.name + '].[' + t.name + '] DROP CONSTRAINT [' + uc.name + '];' AS DropStatement
FROM
    sys.tables t
JOIN
    sys.schemas sch ON sch.schema_id = t.schema_id
JOIN
    sys.indexes pk ON t.object_id = pk.object_id AND pk.is_primary_key = 1
JOIN
    sys.index_columns pkc ON pk.object_id = pkc.object_id AND pk.index_id = pkc.index_id
JOIN
    sys.columns c ON c.object_id = pk.object_id AND c.column_id = pkc.column_id
JOIN
    sys.indexes uc ON uc.object_id = t.object_id AND uc.is_unique_constraint = 1
JOIN
    sys.index_columns ucc ON uc.object_id = ucc.object_id AND uc.index_id = ucc.index_id AND ucc.column_id = pkc.column_id
WHERE
    -- Ensure both constraints are on the same column(s)
    pkc.key_ordinal = ucc.key_ordinal
    -- Prevent listing same index
    AND pk.index_id <> uc.index_id
    and t.name = 'EmpTbl'
ORDER BY
    sch.name, t.name, uc.name;
```

Output:

This query will return results like:

SchemaName	TableName	UniqueConstraintName	DropStatement
HR	EmpTbl	EmpTbl_id_uindex	ALTER TABLE [HR].[EmpTbl] DROP CONSTRAINT [EmpTbl_id_uindex];

You can **copy-paste and run** these drop statements manually **after careful review**.

Safety Tips Before Dropping:

-  Double-check that these constraints are not being used by **foreign keys, triggers, or application code**.
-  Optionally, **script out the constraint definitions** first, in case you want to restore them later.
-  Perform changes in a **test/staging environment first**, if possible.

<https://www.sqldbachamps.com>